



ATESTAT PROFESIONAL
INFORMATICĂ

Maze Game

Îndrumător:
Prof. Maier Cornelia

Elev:
Holerga Andrei-Rareș
XII Informatică

2026

Cuprins

Cuprins.....	2
Introducere. Motivația alegerii	3
Prezentarea generală a temei	3
Aspecte teoretice	4
Despre limbajul C# și platforma .NET.....	4
Despre Visual Studio	4
Descrierea aplicației.....	5
Interfața aplicației.....	5
Meniul principal (Main menu).....	5
Jocul propriu-zis	5
Meniul de joc pierdut (game over)	7
Funcționalitatea aplicației.....	7
Ciclul de joc	7
Generarea labirintului.....	9
Mișcarea entităților și avatarelor	10
Inteligența artificială a inamicilor.....	11
Condițiile de câștig și pierdere	11
Detectarea apăsării tastelor.....	11
Modificarea dificultății în funcție de nivel	12
Desenarea jocului pe ecran	12
Salvarea scorului maxim	13
Bibliografie	13

Introducere. Motivația alegerii

Lumea jocurilor video a reprezentat întotdeauna o sursă de fascinație și inspirație pentru generații întregi de tineri pasionați de tehnologie. Încă de la primele titluri arcade, jocurile au combinat logica matematică cu creativitatea vizuală, oferind utilizatorilor experiențe interactive captivante. Tocmai această intersecție dintre programare și divertisment m-a determinat să aleg ca temă pentru atestatul profesional un joc de tip labirint, Maze Game.

Motivația principală a alegerii acestui proiect a fost dorința de a aprofunda concepte fundamentale din informatică, precum algoritmi de generare procedurală, algoritmi de pathfinding și principiile programării orientate pe obiecte, aplicându-le într-un context practic și vizibil. Un joc reprezintă un mediu ideal de învățare: fiecare funcționalitate implementată are un efect imediat și observabil, ceea ce ușurează înțelegerea și depanarea codului.

De asemenea, am dorit să explorez platforma .NET și limbajul C# într-un proiect concret, dincolo de exercițiile teoretice. Dezvoltarea unei aplicații Windows Forms complete, de la interfața grafică până la logica internă a jocului, mi-a oferit o perspectivă de ansamblu asupra procesului de creare a unui produs software funcțional.

Nu în ultimul rând, jocurile de tip labirint au o tradiție îndelungată în istoria informaticii, de la primele implementări pe computere mainframe până la titlurile moderne, ceea ce le face un subiect clasic, bine documentat, dar în același timp suficient de flexibil pentru a permite inovație și personalizare.

Prezentarea generală a temei

Maze Game este o aplicație de tip joc video, dezvoltată în C# utilizând platforma .NET și mediul de dezvoltare Visual Studio, sub forma unei aplicații Windows Forms.

Jocul presupune navigarea unui personaj (reprezentat printr-un avatar de iepure) printr-un labirint generat aleatoriu, cu scopul de a ajunge la ieșirea acestuia, evitând în același timp inamicii (reprezenți prin avatare de lup) care îl urmăresc activ. Pe măsură ce jucătorul avansează în niveluri, labirintul devine mai complex, iar inamicii devin mai rapizi și mai numeroși, crescând astfel dificultatea jocului.

Principalele componente tehnice ale aplicației sunt:

- **Generarea procedurală a labirintului** - folosind un algoritm Breadth First Search (BFS) și introducerea de cicluri pentru a crea multiple soluții, garantând că jocul este întotdeauna rezolvabil;
- **Inteligența artificială a inamicilor** - bazată tot pe algoritmul BFS, inamicii calculează în permanență cel mai scurt drum spre jucător;
- **Interpolarea mișcării (lerp)** - pentru o animație fluidă a avatarelor pe ecran;
- **Sistemul de dificultate progresivă** - viteza inamicilor, dimensiunea labirintului și densitatea pereților se modifică odată cu avansarea în niveluri;
- **Salvarea scorului maxim** - persistența datelor între sesiuni prin scriere în fișier.

Aplicația gestionează trei stări principale: meniul principal, jocul propriu-zis și ecranul de game over, fiecare cu propria interfață și logică.

Aspecte teoretice

Despre limbajul C# și platforma .NET

C# (pronunțat „C sharp”) este un limbaj de programare modern, orientat pe obiecte, dezvoltat de Microsoft și lansat în anul 2000 ca parte a platformei .NET. A fost conceput de Anders Hejlsberg și echipa sa cu scopul de a combina puterea și eficiența limbajului C++ cu simplitatea și siguranța limbajului Java, rezultând un limbaj robust, expresiv și ușor de utilizat.

Limbajul C# suportă toate paradigmele majore ale programării moderne: programarea orientată pe obiecte (încapsulare, moștenire, polimorfism), programarea funcțională (expresii lambda, LINQ), programarea asincronă (async/await) și programarea generică. Tipizarea sa statică și puternică reduce erorile de execuție, iar sistemul de gestionare automată a memoriei (Garbage Collector) elimină necesitatea gestionării manuale a acesteia.

Platforma .NET este un framework de dezvoltare software creat de Microsoft, care oferă un mediu de execuție comun (CLR - Common Language Runtime) și o bibliotecă vastă de clase (BCL - Base Class Library). Codul scris în C# este compilat în bytecode intermediar (IL - Intermediate Language), care este apoi interpretat și optimizat de CLR la momentul execuției, printr-un proces numit JIT (Just-In-Time compilation). Acest mecanism asigură portabilitate și performanță ridicată.

În contextul acestui proiect, platforma .NET pune la dispoziție biblioteca Windows Forms, care permite crearea de interfețe grafice pentru aplicații desktop Windows. Windows Forms oferă componente vizuale predefinite (butoane, etichete, panouri) și un sistem de evenimente (event-driven programming) care simplifică considerabil dezvoltarea aplicațiilor interactive. Pentru desenarea graficelor în joc, s-a folosit namespace-ul System.Drawing, parte integrantă a bibliotecii .NET.

Despre Visual Studio

Visual Studio este cel mai utilizat mediu integrat de dezvoltare (IDE - Integrated Development Environment) pentru platforma .NET și limbajul C#, dezvoltat de Microsoft. Acesta oferă un set complet de instrumente necesare în procesul de dezvoltare software: editor de cod cu evidențierea sintaxei și autocompletare inteligentă (IntelliSense), designer vizual pentru interfețe Windows Forms (drag-and-drop), debugger integrat pentru depistarea și corectarea erorilor, și manager de pachete NuGet pentru adăugarea de biblioteci externe.

Designerul vizual din Visual Studio a fost esențial în construirea interfeței aplicației, permițând plasarea și configurarea componentelor grafice (label-uri, butoane, panel-uri) direct pe suprafața formularului, fără a necesita scrierea manuală a codului de inițializare. Debugger-ul integrat a facilitat identificarea erorilor de logică din algoritmi de generare a labirintului și de mișcare a entităților, permițând executarea pas cu pas a codului și inspectarea valorilor variabilelor în timp real.

Descrierea aplicației

Interfața aplicației

Această aplicație conține un singur form al cărui layout se schimbă în funcție de starea jocului, meniu sau jocul propriu-zis.

Meniul principal (Main menu)



Acesta este meniul care apare atunci când utilizatorul deschide aplicația, dar și când acesta alege să se întoarcă la meniu după ce a pierdut.

Designul este simplu, un label pentru titlu, un label pentru autorul programului și un buton de start cu textul „Play”. Tema de culori este formată din două culori: mov (#1d0056) și alb (#ffffff).

Jocul propriu-zis



După apăsarea butonului de Start, utilizatorul va vedea ecranul de mai sus, care reprezintă suprafața de joc.



În partea din stânga se regăsesc:

- Numărul nivelului (Level)
- Cel mai bun scor, salvat în fișier (Highscore)
- Instrucțiuni de joc în limba engleză (How to play)



În partea din dreapta se regăsesc instrucțiuni privind controlul personajului din joc: se folosesc tastele W, A, S și D pentru selectarea direcției personajului:

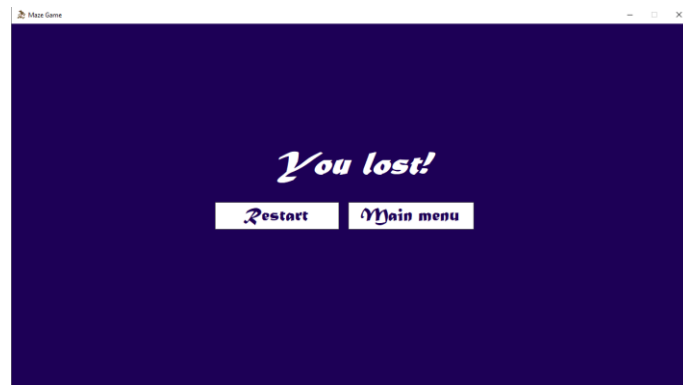
- W - SUS
- A - STÂNGA
- S - JOS
- D - DREAPTA



În centrul ecranului se regăsesc:

- un labirint generat automat, aleatoriu
- avatarul jucătorului (un iepure)
- avatarul inamicului (un lup)

Meniul de joc pierdut (game over)



Acest meniul apare atunci când jucătorul a pierdut jocul. Utilizatorul are două opțiuni:

- Restart (se reîncepe jocul, jucătorul este trimis direct la jocul propriu zis)
- Main menu (jucătorul este trimis la meniul principal)

Funcționalitatea aplicației

Ciclul de joc

Jocul funcționează folosind ciclul (game loop), fiecare iterație fiind un „tick” al unui timer. Pentru fiecare tick:

```
private void gameLoopTimer_Tick(object sender, EventArgs e)
{
```

1. Se verifică dacă jocul s-a terminat

```
    if (gameRunning)
    {
```

2. Se verifică dacă se îndeplinesc condițiile de pierdere sau câștig

```
        if (enemies[0].position == player.position)
            LevelLost();
        if (enemies[1].position == player.position && level >= enemy2level)
            LevelLost();

        if (player.lastPos == new Point(gridSize - 1, gridSize - 1))
        {
            LevelWon();
        }
    }
```

3. Se mișcă entitățile

```
if (tick % player.gameTicksPerMove == 0)
{
    Move(player);
}

if (tick % enemies[0].gameTicksPerMove == 0)
{
    if (enemies[0].position != player.position)
        enemies[0].direction = AiBFS(enemies[0].position, player.position);

    Move(enemies[0]);
}

if (tick % enemies[1].gameTicksPerMove == 0 && level >= enemy2level)
{
    if (enemies[1].position != player.position)
        enemies[1].direction = AiBFS(enemies[1].position, player.position);

    Move(enemies[1]);
}
```

4. Se afișează mișcarea entităților

```
playerImage.Location = Lerp(player.lastPos, player.position, tick % player.gameTicksPerMove,
    player.gameTicksPerMove, cellSize, xOffset, yOffset);
enemyImage.Location = Lerp(enemies[0].lastPos, enemies[0].position, tick % enemies[0].gameTicksPerMove,
    enemies[0].gameTicksPerMove, cellSize, xOffset, yOffset);

if (level >= enemy2level)
{
    enemyImage2.Location = Lerp(enemies[1].lastPos, enemies[1].position, tick % enemies[1].gameTicksPerMove,
        enemies[1].gameTicksPerMove, cellSize, xOffset, yOffset);
    if (enemies[0].position == enemies[1].position)
    {
        enemyImage.Visible = false;
        enemyImage2.Visible = false;
        enemyGroupImage.Location = enemyImage.Location;
        enemyGroupImage.Visible = true;
    }
    else
    {
        enemyImage.Visible = true;
        enemyImage2.Visible = true;
        enemyGroupImage.Visible = false;
    }
}
```

Generarea labirintului

Jocul are nevoie de un labirint care să aibă mai mult de o soluție pentru ca jucătorul să aibă posibilitatea de a ajunge la ieșire (EXIT) fără a se intersecta cu inamicul. Astfel, generarea automată și aleatorie a labirintului se execută în 2 faze:

1. Generarea unui labirint cu o singură soluție:
 - a. se generează o matrice care conține doar valoarea 1 (pereți)

```
// Initializarea unui labirint
2 references
private void GenerateMaze()
{
    maze = new int[gridSize, gridSize];

    // La inceput in labirint se afla doar pereti
    for (int i = 0; i < gridSize; i++)
    {
        for (int j = 0; j < gridSize; j++) maze[i, j] = 1;
    }
}
```

- b. se parcurge matricea pornind de la punctul (0,0) cu un algoritm Breadth First (parcuregere în lățime) care generează un arbore și se modifică valoarea de pe pozițiile parcurse la 0 (spațiu)

```
private void RemoveWall(int i, int j)
{
    maze[i, j] = 0;
    // Se creeza o lista cu cele 4 directii
    var directions = new List<int di, int dj>
    {
        (0, -2), (0, 2), (-2, 0), (2, 0)
    };
    Randomize(directions);

    // Se verifica fiecare directie si se creeza un drum daca se gaseste un perete
    foreach (var (di, dj) in directions)
    {
        int ni = i + di, nj = j + dj;
        if (InBounds(ni, nj) && maze[ni, nj] == 1)
        {
            maze[i + di / 2, j + dj / 2] = 0;
            RemoveWall(i + di, j + dj);
        }
    }
}
```

2. Se introduc cicluri în arbore modificând o parte din pereți în spațiu, astfel creând mai multe soluții

```
private void MakeLoops(double probability)
{
    for (int i = 1; i < gridSize - 1; i++)
    {
        for (int j = 1; j < gridSize - 1; j++)
        {
            if (maze[i, j] == 1)
            {
                if ((maze[i - 1, j] == 0 && maze[i + 1, j] == 0) ||
                    (maze[i, j - 1] == 0 && maze[i, j + 1] == 0))
                {
                    if (i < gridSize / 3 && j < gridSize / 3)
                    {
                        if (rand.NextDouble() < (probability + spawnProbAdd)) maze[i, j] = 0;
                    }
                    else
                    {
                        if (rand.NextDouble() < probability) maze[i, j] = 0;
                    }
                }
            }
        }
    }
}
```

Labirintul devine tot mai mare și tot mai încărcat (mai puține cicluri create) în funcție de nivel.

În colțul din stânga sus al labirintului (locul în care apare personajul) probabilitatea de a modifica un perete în spațiu este mai mare pentru a nu bloca jucătorul în partea de început.

Mișcarea entităților și avatarelor

Fiecare entitate are o variabilă numită `gameTicksPerMove` care reprezintă numărul de tick-uri care trebuie să treacă pentru ca entitatea să se miște, astfel această reprezintă viteza entității.

Entitățile au o mișcare de tip „grid”, poziția lor fiind reprezentată de indici ai matricei de joc. Dacă această mișcare ar fi transpusă direct pe ecran, avatarele ar părea că „sar” de la o poziție la alta. Această mișcare nu este plăcută vizual, astfel, se folosește un algoritm de tip „lerp” pentru a interpola mișcarea. La fiecare tick al jocului, funcția „Lerp” găsește un punct intermediar între poziția veche și poziția nouă în funcție de tick-ul curent și variabila `gameTicksPerMove` a entității.

```
private bool Move(Entity e)
{
    Point newPos = new Point(e.position.X + e.direction.X, e.position.Y + e.direction.Y);

    if (InBounds(newPos.X, newPos.Y) && maze[newPos.X, newPos.Y] != 1)
    {
        e.lastPos = e.position;
        e.position = newPos;

        return true;
    }

    e.lastPos = e.position;
    return false;
}
```

Mișcarea pe grid

```
private Point Lerp(Point A, Point B, int current, int max, int cellSize, int xOffset, int yOffset)
{
    double t = (double)current / max;

    double lerpedX = A.X + (B.X - A.X) * t;
    double lerpedY = A.Y + (B.Y - A.Y) * t;

    int pixelX = xOffset + (int)(lerpedX * cellSize);
    int pixelY = yOffset + (int)(lerpedY * cellSize);

    return new Point(pixelX, pixelY);
}
```

Funcția de interpolare a mișcării

Inteligența artificială a inamicilor

Inamicii folosesc un algoritm de pathfinding (Breadth First Search/căutare în lățime) pentru a găsi drumul cel mai scurt către personaj.

Condițiile de câștig și pierdere

Pentru câștig, se verifică dacă entitatea personajului a ajuns la finalul labirintului, la poziția [gridSize, gridSize].

Pentru pierdere, se verifică dacă poziția personajului coincide cu poziția unuia dintre inamici.

Detectarea apăsării tastelor

Pentru a controla mișcare personajului, jucătorul folosește tastatura. Detectarea apăsării tastelor se folosește folosind funcția [Form_name]_KeyDown (în cazul acesta: „Form1_KeyDown”).

```
private void Form1_KeyDown(object sender, KeyEventArgs e)
{
    if (gameStarted && !nextLvlButton.Visible)
    {
        if (e.KeyCode == Keys.W)
        {
            if (!gameRunning)
            {
                tick = 0;
                gameRunning = true;
                gameLoopTimer.Enabled = true;
            }
            Point newPos = new Point(player.position.X + 0, player.position.Y - 1);
            if (InBounds(newPos.X, newPos.Y) && maze[newPos.X, newPos.Y] != 1)
                player.direction = new Point(0, -1);
        }
    }
}
```

Modificarea dificultății în funcție de nivel

- Pentru nivelele 1-30: se scade probabilitatea de ștergere a unui perete cu 1%.
- Pentru nivelele 1-5: gameTicksPerMove al inamicilor scade cu 1 (inamicii devin mai rapizi).
- Pentru nivelele divizibile cu 3, până la 15: gameTicksPerMove al inamicilor scade cu 1.
- La nivelul 15 apare al doilea inamic.
- Atunci când viteza personajului este mai mică decât viteza inamicilor: viteza personajului devine egală cu cea a inamicilor.

```
level++;
if (level <= 30) prob -= 0.01;
if (level % 3 == 0 && level <= 15) gridSize += 2;
if (level <= 5)
{
    enemies[0].gameTicksPerMove -= 1;
}
else if (level <= 15 && level % 3 == 0)
{
    enemies[0].gameTicksPerMove -= 1;
}
if (player.gameTicksPerMove > enemies[0].gameTicksPerMove) player.gameTicksPerMove = enemies[0].gameTicksPerMove;
```

Desenarea jocului pe ecran

Pentru a desena jocul pe ecran, se folosește un panel cu funcția paint.

```
private void panel1_Paint(object sender, PaintEventArgs e)
{
    Graphics g = e.Graphics;
    g.Clear(BackColor);
    int cdo = cellSize / 5;

    // Desenez baza tablei
    g.FillRectangle(new SolidBrush(Color.FromArgb(200, 200, 200)),
        new Rectangle(xOffset - cellSize, yOffset - cellSize,
            cellSize * (gridSize + 2), cellSize * (gridSize + 2)));

    // Desenez iesirea din labirint
    g.FillRectangle(new SolidBrush(BackColor),
        new Rectangle(xOffset + cellSize * gridSize - cdo, yOffset + cellSize * (gridSize - 1) - cdo,
            cellSize + cdo * 2, cellSize + cdo * 2));
    g.FillRectangle(new SolidBrush(BackColor),
        new Rectangle(xOffset + cellSize * gridSize - cdo, yOffset + cellSize * gridSize - cdo,
            cellSize + cdo * 2, cellSize + cdo * 2));
    g.FillRectangle(new SolidBrush(BackColor),
        new Rectangle(xOffset + cellSize * (gridSize - 1) - cdo, yOffset + cellSize * gridSize - cdo,
            cellSize + cdo * 2, cellSize + cdo * 2));

    for (int i = 0; i < gridSize; i++)
    {
        for (int j = 0; j < gridSize; j++)
        {
            if (maze[i, j] == 0)
            {
                g.FillRectangle(new SolidBrush(BackColor),
                    new Rectangle(xOffset + i * cellSize - cdo, yOffset + j * cellSize - cdo,
                        cellSize + cdo * 2, cellSize + cdo * 2));
            }
        }
    }
}
```

Salvarea scorului maxim

Pentru a reține scorul maxim atins de jucător, se folosește un fișier în care este scris scorul maxim la închiderea aplicației și din care se citește scorul maxim la deschiderea aplicației.

```
1 reference
private void SaveHighScore(int score)
{
    File.WriteAllText(filePath, score.ToString());
}

1 reference
private int LoadHighScore()
{
    if (File.Exists(filePath))
    {
        string content = File.ReadAllText(filePath);
        int score;

        if (int.TryParse(content, out score))
            return score;
    }

    return 0;
}
```

Bibliografie

- [1] Microsoft Corporation, *Documentație C# și .NET*, disponibil la: <https://learn.microsoft.com/en-us/dotnet/csharp/>.
- [2] Microsoft Corporation, *Windows Forms documentation*, disponibil la: <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/>.
- [3] Wikipedia, *Breadth-first search*, disponibil la: https://en.wikipedia.org/wiki/Breadth-first_search.
- [4] Wikipedia, *Maze generation algorithm*, disponibil la: https://en.wikipedia.org/wiki/Maze_generation_algorithm.
- [5] Wikipedia, *Linear interpolation*, disponibil la: https://en.wikipedia.org/wiki/Linear_interpolation.